Guía de estudio

Formulario "Bienvenidos al campus" en Flutter

Esta guía explica **todo el código** del proyecto **flutter_practica** (archivos `main.dart` y `app.dart`), parte por parte, para que lo puedas escribir de memoria en el examen.

Cómo usarla:

- 1) Mirá el **mapa** (sección 1): es el esqueleto, si te lo acordás, todo lo demás se cuelga de ahí.
- 2) Leé cada sección con su código y su explicación.
- 3) Usá los recuadros amarillos (trucos) y rojos (errores típicos).
- 4) Al final hay preguntas de examen y una **práctica para escribirlo de memoria**.

1. El mapa: cómo está armada la app

Toda la app es un **árbol de widgets** (cajas dentro de cajas). Este es el árbol completo:

```
main()
 `-- runApp( MainApp )                    <- main.dart
      `-- MaterialApp                     <- app.dart (MainApp, StatelessWidget)
           `-- home: MyHomePage           <- StatefulWidget (tiene estado)
                `-- Scaffold
                     |-- appBar: AppBar  ("Bienvenidos al campus")
                     `-- body: SingleChildScrollView      (scroll)
                          `-- Column
                               |-- FadeInImage            (imagen con gif de carga)
                               |-- Padding (50)
                               |    `-- Column
                               |         |-- _inputNombre()
                               |         |-- Divider()
                               |         |-- _inputEmail()
                               |         |-- Divider()
                               |         |-- _inputPassword()
                               |         |-- Divider()
                               |         |-- _crearFecha(context)   -> abre _selectDate()
                               |         |-- Divider()
                               |         `-- _crearDropdown()      -> usa getOpcionesDropdown()
                               `-- ElevatedButton "Iniciar Sesion" -> _mostrarAlerta()
```

TRUCO PARA RECORDAR: El orden de los campos es **N-E-P-F-D**: **N**ombre, **E**mail, **P**assword, **F**echa, **D**ropdown. Frase: *"**N**unca **E**studio **P**ara **F**lutter **D**ormido"*. Entre cada uno va un `Divider()` (4 en total).

Las 3 clases del archivo app.dart:

Clase	Tipo	Para qué sirve
`MainApp`	StatelessWidget	Configura la app (MaterialApp) y dice qué pantalla abrir
`MyHomePage`	StatefulWidget	La pantalla. Solo crea su State
`_MyHomePageState`	State<MyHomePage>	Variables + build() + todos los métodos

Los 8 métodos dentro de `_MyHomePageState` (en orden):

- `build` - dibuja la pantalla
- `_crearFecha` - TextField de la fecha
- `_selectDate` - abre el calendario (async)
- `_inputNombre`, `_inputEmail`, `_inputPassword` - los 3 TextField de texto
- `getOpcionesDropdown` - arma la lista de opciones
- `_crearDropdown` - el desplegable de profesiones
- `_mostrarAlerta` - el cartel (AlertDialog) al apretar el botón

2. main.dart - el punto de entrada

```
import 'package:flutter/material.dart';
import 'package:flutter_practica/src/app.dart';

void main() {
  runApp(const MainApp());
}
```

- `import 'package:flutter/material.dart';` trae todos los widgets de Material (Text, Scaffold, etc.).
- `import 'package:flutter_practica/src/app.dart';` trae la clase `MainApp`. El formato es **package:NOMBRE_DEL_PROYECTO/ruta** (sin la carpeta lib).
- `void main()` es lo primero que se ejecuta.
- `runApp(const MainApp())` arranca Flutter con el widget `MainApp`.

3. MainApp - la configuración

```
import 'package:flutter/material.dart';

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Practica',
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const MyHomePage(),
    );
  }
}
```

- `extends StatelessWidget`: no cambia nunca, por eso es Stateless (sin estado).
- `const MainApp({super.key});` es el constructor. Siempre lleva `{super.key}`.
- `@override Widget build(BuildContext context)`: método obligatorio que devuelve lo que se dibuja.
- `MaterialApp`: la base de toda app Material.
- `debugShowCheckedModeBanner: false` saca la cinta roja "DEBUG" de la esquina.
- `title`: nombre de la app. `theme: ThemeData(colorSchemeSeed: Colors.blue)`: colores en base al azul.
- `home: const MyHomePage()`: **la pantalla que se abre primero**.

OJO: ¿Por qué MyHomePage es una clase aparte y no está todo adentro de MaterialApp? Porque `showDatePicker` y `showDialog` necesitan un **context que esté DEBAJO del MaterialApp**. Si pusieras el Scaffold directo en MainApp, el calendario y la alerta tirarían error.

4. MyHomePage y sus variables (el estado)

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  String _nombre = '';
```

```
String _fecha = '';
String _email = '';
final List<String> _profesiones = [
  'Sin Dato',
  'Doctor',
  'Medico',
  'Profesor',
  'Gomero',
  'Cartonero',
];
final TextEditingController _inputFieldController = TextEditingController();
String _opcionseleccionada = 'Sin Dato';
```

- Un **StatefulWidget** tiene **2 clases**: el widget (`MyHomePage`) y su estado (`_MyHomePageState`).
- `createState() => _MyHomePageState();` conecta el widget con su estado.
- El guion bajo `_` significa **privado** (solo se usa dentro de este archivo).
- `_nombre`, `_email`, `_fecha`: guardan lo que escribe el usuario. Arrancan vacías `''`.
- `_profesiones`: lista de las 6 opciones del dropdown.
- `_inputFieldController`: un **TextEditingController** que permite **escribir texto en el TextField desde el código** (lo usamos para poner la fecha elegida).
- `_opcionseleccionada = 'Sin Dato'`: la opción que muestra el dropdown al empezar. **Tiene que ser igual a uno de los elementos de la lista.**

TRUCO PARA RECORDAR: Profesiones (6): **Sin Dato, Doctor, Medico, Profesor, Gomero, Cartonero**. Variables en orden: **nombre, fecha, email** (Strings) -> **profesiones** (List) -> **controller** -> **opcionseleccionada**.

5. build() - la pantalla

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      centerTitle: true,
      backgroundColor: Colors.blue,
      title: Text(
        "Bienvenidos al campus",
        style: TextStyle(color: Colors.white),
      ),
    ),
    body: SingleChildScrollView(
      child: Column(
        children: [
          FadeInImage(
            placeholder: AssetImage('assets/loading-img.gif'),
            image: NetworkImage(
              'https://tse4.mm.bing.net/th/id/OIP.
 kJL2KM46ggKzXdFkEfHDuAHaDz?r=0&rs=1&pid=ImgDetMain&o=7&rm=3',
            ),
            width: 350,
            height: 200,
          ),
          Padding(
            padding: EdgeInsets.all(50),
            child: Column(
              children: [
                _inputNombre(),
                Divider(),
                _inputEmail(),
                Divider(),
                _inputPassword(),
                Divider(),
                _crearFecha(context),
                Divider(),
                _crearDropdown(),
              ],
            ),
          ),
          ElevatedButton(
            onPressed: _mostrarAlerta,
            child: Text("Iniciar Sesion"),
          ),
        ],
      ),
    ),
  );
}
```

AppBar (la barra de arriba)

- `centerTitle: true` centra el título. `backgroundColor: Colors.blue` fondo azul.
- `title: Text(..., style: TextStyle(color: Colors.white))` texto blanco.

body

- `SingleChildScrollView`: permite **hacer scroll**. Sin esto, al abrir el teclado aparece la franja amarilla y negra (overflow). Lleva **child** (uno solo).
- `Column`: pone los widgets uno **debajo** del otro. Lleva **children** (lista).
- `FadeInImage`: muestra el `placeholder` (gif local, `AssetImage`) mientras carga la `image` de internet (`NetworkImage`), y después hace un fundido.
- `Padding(padding: EdgeInsets.all(50))`: deja 50 de margen en los 4 lados alrededor de los campos.

- `ElevatedButton`: botón. `onPressed: _mostrarAlerta` (sin paréntesis: le pasamos la función, no la ejecutamos).

TRUCO PARA RECORDAR: child = un solo hijo (Padding, SingleChildScrollView, Expanded, ElevatedButton). **children** = lista de hijos (Column, Row).

OJO: El gif `assets/loading-img.gif` tiene que estar declarado en **pubspec.yaml**:

```
flutter:
 assets:
  - assets/loading-img.gif
```

6. Los TextField de texto

Los 3 tienen la **misma receta**:

```
TextField(
  <propiedad especial>,
  decoration: InputDecoration(
    label: Text("..."),     // el título del campo
    hint: Text("..."),      // texto gris de ayuda
    icon: Icon(Icons....),  // ícono a la izquierda
  ),
  onChanged: (value) => { setState(() { _variable = value; }) },  // nombre y email
)
```

_inputNombre()

```
  Widget _inputNombre() {
    return TextField(
      textCapitalization: TextCapitalization.sentences,
      decoration: InputDecoration(
        label: Text("Nombre completo"),
        hint: Text("Escriba su Nombre"),
        icon: Icon(Icons.accessibility),
        suffixIcon: Icon(Icons.supervised_user_circle),
      ),
      onChanged: (value) => {
        setState(() {
          _nombre = value;
        }),
      },
    );
  }
```

- `textCapitalization: TextCapitalization.sentences`: pone mayúscula a la primera letra.
- `suffixIcon`: ícono a la **derecha** (adentro del campo). Es el único que lo tiene.
- `onChanged`: se ejecuta **cada vez que se escribe una letra**; guarda el texto en `_nombre`.

_inputEmail()

```
  Widget _inputEmail() {
    return TextField(
      keyboardType: TextInputType.emailAddress,
      decoration: InputDecoration(
        label: Text("Email"),
        hint: Text("Escriba su Email"),
        icon: Icon(Icons.email),
      ),
      onChanged: (value) => {
        setState(() {
          _email = value;
        }),
      },
    );
  }
```

- `keyboardType: TextInputType.emailAddress`: muestra el teclado con @.

_inputPassword()

```
  Widget _inputPassword() {
    return TextField(
      obscureText: true,
      decoration: InputDecoration(
        label: Text("Password"),
        hint: Text("Escriba su Password"),
        icon: Icon(Icons.password),
      ),
    );
  }
```

- `obscureText: true`: oculta lo que escribís con puntitos. **No tiene onChanged** (la contraseña no se guarda).

TRUCO PARA RECORDAR: Propiedad especial de cada uno: Nombre -> **textCapitalization**, Email -> **keyboardType**, Password -> **obscureText**. Íconos: **accessibility** (+ suffix supervised_user_circle), **email**, **password**.

7. La fecha: _crearFecha() y _selectDate()

```
Widget _crearFecha(BuildContext context) {
  return TextField(
    enableInteractiveSelection: true,
    controller: _inputFieldController,
    decoration: InputDecoration(
      label: Text("Fecha"),
      hint: Text("Fecha"),
      icon: Icon(Icons.date_range),
    ),
    onTap: () {
      FocusScope.of(context).requestFocus(FocusNode());
      _selectDate(context);
    },
  );
}

Future<DateTime?> _selectDate(BuildContext content) async {
  DateTime? calendario = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(2022),
    lastDate: DateTime(2030),
  );
  if (calendario != null) {
    setState(() {
      _fecha = calendario.toString().substring(0, 10);
      _inputFieldController.text = _fecha;
    });
  }
  return calendario;
}
```

_crearFecha

- `controller: _inputFieldController`: así después podemos escribir la fecha adentro del campo.
- `onTap`: se ejecuta al **tocar** el campo.
- `FocusScope.of(context).requestFocus(FocusNode());`: le saca el foco al campo para que **no se abra el teclado**.
- Después llama a `_selectDate(context)` que abre el calendario.

_selectDate

- `Future<DateTime?>` + `async`: es una función que **espera** algo (que el usuario elija la fecha).
- `await showDatePicker(...)`: abre el calendario y espera. Devuelve la fecha o `null` si cancelan.
- `initialDate` = hoy (`DateTime.now()`), `firstDate` = 2022, `lastDate` = 2030.
- `if (calendario != null)`: solo si eligió una fecha.
- `calendario.toString().substring(0, 10)`: la fecha completa es *2026-10-09 00:00:00.000*; con substring(0,10) nos quedamos con **2026-10-09** (los primeros 10 caracteres).
- `_inputFieldController.text = _fecha;`: escribe la fecha en el TextField.
- Todo dentro de `setState` para que la pantalla se actualice.

OJO: El parámetro se llama `content` (no context) pero adentro se usa `context`. Funciona igual porque todo State ya tiene su propio `context`. Si lo escribís como `BuildContext context` también está bien.

8. El Dropdown: getOpcionesDropdown() y _crearDropdown()

```
List<DropdownMenuItem<String>> getOpcionesDropdown() {
  List<DropdownMenuItem<String>> lista = [];
  for (var profesion in _profesiones) {
    lista.add(DropdownMenuItem(value: profesion, child: Text(profesion)));
  }
  return lista;
}

Widget _crearDropdown() {
  return Row(
    children: <Widget>[
      Icon(Icons.select_all),
      SizedBox(width: 30.0),
      Expanded(
        child: DropdownButton(
          //Valor por defecto del dropdown
          value: _opcionseleccionada,
          items: getOpcionesDropdown(),
          onChanged: (opt) {
            setState(() {
              _opcionseleccionada = opt.toString();
            });
          },
        ),
      ),
    ],
  );
}
```

- `getOpcionesDropdown` recorre `_profesiones` con un **for-in** y por cada profesión crea un `DropdownMenuItem(value: profesion, child: Text(profesion))`. Devuelve la lista.
- `_crearDropdown` devuelve una `Row` (fila, de izquierda a derecha) con: ícono `select_all` -> espacio `SizedBox(width: 30.0)` -> `Expanded` con el dropdown.
- `Expanded`: hace que el dropdown ocupe todo el ancho que sobra.
- `value`: la opción que se ve elegida. `items`: la lista de opciones.
- `onChanged: (opt) {...}`: cuando eligen otra, se guarda en `_opcionseleccionada` dentro de setState.

TRUCO PARA RECORDAR: Row = **I-S-E**: **I**cono, **S**izedBox, **E**xpanded. Dropdown = **V-I-O**: **V**alue, **I**tems, **O**nChanged.

9. La alerta: _mostrarAlerta()

```
void _mostrarAlerta() {
  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text("Bienvenido"),
        content: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          mainAxisSize: MainAxisSize.min,
          children: <Widget>[
            Text("Bienvenido al campus, Registro hecho con exito"),
            Text("Nombre: $_nombre"),
            Text("Email: $_email"),
          ],
        ),
        actions: <Widget>[
          TextButton(
            onPressed: () => {
              setState(() {
                Navigator.of(context).pop();
              }),
            },
            child: Text("Cancelar"),
          ),
          TextButton(
            onPressed: () => {
              setState(() {
                Navigator.of(context).pop();
              }),
            },
            child: Text("Aceptar"),
          ),
        ],
      );
    },
  );
}
}
```

- `showDialog(context: context, builder: (context) { return AlertDialog(...); })`: muestra un cartel encima de la pantalla.
- `AlertDialog` tiene 3 partes: **title**, **content** y **actions** (los botones).
- En `content` va una Column con `mainAxisSize: MainAxisSize.min` para que el cartel **ocupe solo lo necesario** (si no, se estira a toda la pantalla).
- `"Nombre: $_nombre"`: el `$` mete el valor de la variable dentro del texto (interpolación).
- Los 2 `TextButton` (Cancelar y Aceptar) hacen lo mismo: `Navigator.of(context).pop()` = **cerrar el cartel**.

TRUCO PARA RECORDAR: AlertDialog = **T-C-A**: **T**itle, **C**ontent, **A**ctions. El contenido muestra 3 Text: mensaje de bienvenida, Nombre y Email (la fecha y la profesión NO se muestran).

OJO: En `onChanged: (value) => { ... }` y en los botones se usa `=> { }`. Funciona, pero lo más correcto en Dart es `(value) { setState(...); }` (sin la flecha). Si en el examen escribís cualquiera de las dos, anda.

10. Preguntas típicas de examen

P: ¿Diferencia entre StatelessWidget y StatefulWidget?

R: Stateless no cambia (MainApp). Stateful tiene datos que cambian y se redibuja con setState (MyHomePage).

P: ¿Qué hace setState?

R: Avisa a Flutter que cambió una variable y vuelve a ejecutar build() para actualizar la pantalla.

P: ¿Para qué sirve el TextEditingController?

R: Para leer o escribir el texto de un TextField desde el código. Lo usamos para poner la fecha.

P: ¿Por qué _selectDate es async?

R: Porque showDatePicker devuelve un Future: hay que esperar (await) a que el usuario elija.

P: ¿Para qué está el SingleChildScrollView?

R: Para poder deslizar la pantalla y que no haya overflow cuando aparece el teclado.

P: ¿Qué es FadeInImage?

R: Una imagen que muestra un placeholder (gif local) mientras carga la imagen de internet.

P: ¿Qué hace Navigator.of(context).pop()?

R: Cierra lo que está arriba de todo; en este caso el AlertDialog.

P: ¿Para qué sirve obscureText?

R: Oculta el texto (contraseña).

P: ¿Qué hace Expanded?

R: Hace que su hijo ocupe todo el espacio libre de la Row/Column.

P: ¿Qué significa el _ adelante de un nombre?

R: Que es privado del archivo.

P: ¿Diferencia entre Column y Row?

R: Column apila vertical (uno abajo del otro), Row horizontal (uno al lado del otro).

P: ¿Qué indica home en MaterialApp?

R: La primera pantalla que se muestra al abrir la app.

11. Errores típicos a evitar

- Olvidarse el `setState` -> la variable cambia pero la pantalla no se actualiza.
- Poner en `_opcionseleccionada` un valor que **no** está en la lista -> error rojo del Dropdown.
- No declarar el gif en **pubspec.yaml** -> "Unable to load asset".
- Escribir `onPressed: _mostrarAlerta()` con paréntesis -> se ejecuta solo al dibujar. Va **sin** paréntesis.
- Usar `child` en Column/Row (va `children`) o `children` en Padding (va `child`).
- Olvidarse `@override` arriba de build y createState.
- Olvidarse de cerrar paréntesis y poner las comas al final de cada propiedad.
- Olvidarse del `return` en los métodos que devuelven un Widget.

12. Código completo de app.dart

Para repasar de corrido. Leelo de arriba a abajo pensando en el mapa de la sección 1.

```
import 'package:flutter/material.dart';

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Practica',
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  String _nombre = '';
  String _fecha = '';
  String _email = '';
  final List<String> _profesiones = [
    'Sin Dato',
    'Doctor',
    'Medico',
    'Profesor',
    'Gomero',
    'Cartonero',
  ];
  final TextEditingController _inputFieldController = TextEditingController();
  String _opcionseleccionada = 'Sin Dato';
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        centerTitle: true,
        backgroundColor: Colors.blue,
        title: Text(
          "Bienvenidos al campus",
          style: TextStyle(color: Colors.white),
        ),
      ),
      body: SingleChildScrollView(
        child: Column(
          children: [
            FadeInImage(
              placeholder: AssetImage('assets/loading-img.gif'),
              image: NetworkImage(
                'https://tse4.mm.bing.net/th/id/OIP.
   kJL2KM46ggKzXdFkEfHDuAHaDz?r=0&rs=1&pid=ImgDetMain&o=7&rm=3',
              ),
              width: 350,
              height: 200,
            ),
            Padding(
              padding: EdgeInsets.all(50),
              child: Column(
                children: [
                  _inputNombre(),
                  Divider(),
                  _inputEmail(),
                  Divider(),
                  _inputPassword(),
                  Divider(),
                  _crearFecha(context),
```

```
                Divider(),
                _crearDropdown(),
              ],
            ),
          ),
          ElevatedButton(
            onPressed: _mostrarAlerta,
            child: Text("Iniciar Sesion"),
          ),
        ],
      ),
    ),
  );
}

Widget _crearFecha(BuildContext context) {
  return TextField(
    enableInteractiveSelection: true,
    controller: _inputFieldController,
    decoration: InputDecoration(
      label: Text("Fecha"),
      hint: Text("Fecha"),
      icon: Icon(Icons.date_range),
    ),
    onTap: () {
      FocusScope.of(context).requestFocus(FocusNode());
      _selectDate(context);
    },
  );
}

Future<DateTime?> _selectDate(BuildContext content) async {
  DateTime? calendario = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(2022),
    lastDate: DateTime(2030),
  );
  if (calendario != null) {
    setState(() {
      _fecha = calendario.toString().substring(0, 10);
      _inputFieldController.text = _fecha;
    });
  }
  return calendario;
}

Widget _inputNombre() {
  return TextField(
    textCapitalization: TextCapitalization.sentences,
    decoration: InputDecoration(
      label: Text("Nombre completo"),
      hint: Text("Escriba su Nombre"),
      icon: Icon(Icons.accessibility),
      suffixIcon: Icon(Icons.supervised_user_circle),
    ),
    onChanged: (value) => {
      setState(() {
        _nombre = value;
      }),
    },
  );
}

Widget _inputEmail() {
  return TextField(
    keyboardType: TextInputType.emailAddress,
    decoration: InputDecoration(
      label: Text("Email"),
      hint: Text("Escriba su Email"),
      icon: Icon(Icons.email),
    ),
    onChanged: (value) => {
      setState(() {
        _email = value;
      }),
```

```
    },
  );
}

Widget _inputPassword() {
  return TextField(
    obscureText: true,
    decoration: InputDecoration(
      label: Text("Password"),
      hint: Text("Escriba su Password"),
      icon: Icon(Icons.password),
    ),
  );
}

List<DropdownMenuItem<String>> getOpcionesDropdown() {
  List<DropdownMenuItem<String>> lista = [];
  for (var profesion in _profesiones) {
    lista.add(DropdownMenuItem(value: profesion, child: Text(profesion)));
  }
  return lista;
}

Widget _crearDropdown() {
  return Row(
    children: <Widget>[
      Icon(Icons.select_all),
      SizedBox(width: 30.0),
      Expanded(
        child: DropdownButton(
          //Valor por defecto del dropdown
          value: _opcionseleccionada,
          items: getOpcionesDropdown(),
          onChanged: (opt) {
            setState(() {
              _opcionseleccionada = opt.toString();
            });
          },
        ),
      ),
    ],
  );
}

void _mostrarAlerta() {
  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text("Bienvenido"),
        content: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          mainAxisSize: MainAxisSize.min,
          children: <Widget>[
            Text("Bienvenido al campus, Registro hecho con exito"),
            Text("Nombre: $_nombre"),
            Text("Email: $_email"),
          ],
        ),
        actions: <Widget>[
          TextButton(
            onPressed: () => {
              setState(() {
                Navigator.of(context).pop();
              }),
            },
            child: Text("Cancelar"),
          ),
          TextButton(
            onPressed: () => {
              setState(() {
                Navigator.of(context).pop();
              }),
            },
            child: Text("Aceptar"),
          ),
```

```
            ],
          );
        },
      );
    }
  }
```

13. Práctica: escribilo de memoria

Agarrá una hoja en blanco y escribí todo **sin mirar**. Después corregí con la sección 12. Repetilo hasta que salga sin errores. Seguí este orden:

	Paso	1ra	2da	3ra
1	main.dart: los 2 imports + main() + runApp(const MainApp())			
2	import de material.dart en app.dart			
3	MainApp (Stateless) con MaterialApp: banner false, title, theme, home			
4	MyHomePage (Stateful) + createState			
5	_MyHomePageState: las 6 variables (nombre, fecha, email, profesiones, controller, opcionseleccionada)			
6	build: Scaffold -> AppBar (centerTitle, color, title blanco)			
7	body: SingleChildScrollView -> Column -> FadeInImage			
8	Padding(50) -> Column con N-E-P-F-D y 4 Divider			
9	ElevatedButton "Iniciar Sesion" -> _mostrarAlerta			
10	_crearFecha (controller, decoration, onTap con FocusScope)			
11	_selectDate (async, await showDatePicker, 2022-2030, substring(0,10))			
12	_inputNombre / _inputEmail / _inputPassword (receta del TextField)			
13	getOpcionesDropdown (for-in + DropdownMenuItem)			
14	_crearDropdown (Row: Icono, SizedBox, Expanded -> DropdownButton V-I-O)			
15	_mostrarAlerta (showDialog -> AlertDialog T-C-A, 2 TextButton con pop)			

TRUCO PARA RECORDAR: Estudiá **de a una sección por vez**: primero el mapa, después MainApp, después las variables... Si te trabás en el examen, dibujá primero el árbol de la sección 1 en un costado de la hoja y después completá cada método.